INFYSLATE · SUPPLY CHAIN AI

Control Tower Technical Plan

> Architecture, data model, per-module algorithms, sample-data strategy & Claude-Code build sequence

---

Document: 2 of 3 — Technical Plan
Companion documents: Business Plan & Demo Narrative · UI/UX & Demo Design
Audience: Founder, building solo with Claude Code (planning only — no code in this document)
Date: November 2026

1. Architecture Overview

The demo is one deployable application, not seven. A single full-stack TypeScript app serves the Control Tower home and all seven modules as routes, reading and writing one shared "data spine" and one shared "AI alerts" table. This is deliberate: a solo founder building with Claude Code should never be managing multiple services, multiple deploys or multiple repos for a pitch demo — every extra moving part is a thing that can break five minutes before a pitch.

Layers

Layer	Responsibility
UI (React/Next.js routes)	Control Tower home + 7 module screens, shared component library
Module logic (TypeScript services)	One service per module implementing its algorithm (Section 5)
Alerts engine	Common schema; every module pushes flags here; home screen reads from here
Data spine (SQLite or JSON+Zod)	Shared entities (Section 3); seeded by the sample-data generator (Section 4)

Why a single app, not microservices

- One deploy target (e.g. Vercel) — or fully offline on a laptop for pitch day.
- One codebase Claude Code can hold context on across sprints.
- One data spine makes the "one data spine" pitch claim literally true in the code, not just the slides.
- No network calls between services to fail live on stage.

2. Recommended Tech Stack

Layer	Choice	Why this, for this build
Framework	Next.js (App Router) + TypeScript	Single app for UI + API routes; huge Claude Code training surface; one deploy.
Styling / components	Tailwind CSS + shadcn/ui + lucide-react icons	Fast, consistent component system across 7 modules without a design-engineer; matches UI Plan §3.
Charts	Recharts or Tremor	Covers forecast lines, KPI tiles, heatmaps and sparklines needed across modules with one library.
Data store	SQLite (via better-sqlite3 or Drizzle ORM)	Zero-infra, file-based, resets instantly for demo mode; swappable for Postgres in Phase 1 (Business Plan §9).
Validation / schemas	Zod	One schema definition shared by the data spine, the generator and the UI forms.
Optional LLM calls (DocAI, flag narration)	A single provider behind one thin wrapper, with a cached/offline fallback per sample document	Keeps the "AI" genuinely AI where it adds value, without making demo day depend on live network/API latency.
Forecast/optimization math	Plain TypeScript (no Python service)	Simple statistics (moving average, z-score safety stock, greedy/LP heuristics) are easily implemented in TS; avoids a second runtime for a demo.

CONSTRAINT
Everything must run with `npm run dev` on a single laptop, fully offline, before it is allowed to depend on any hosted deploy. Hosted deploy (e.g. Vercel) is the "send a link to an investor" path, not the primary pitch-day path.

3. Shared Data Model

Every module reads and writes the same core entities. This is the backbone that makes cross-module alerts (Business Plan §5) possible.

Entity	Key fields	Used by
Sku	id, name, category, unit cost, unit price, margin	All modules
Location	id, name, type (warehouse/store/3PL hub), lat/lng (for map views)	3PL, R&P, Transfer, Salvage, OOS
InventoryPosition	skuId, locationId, onHand, onOrder, reserved, asOfDate	R&P, Transfer, Salvage, OOS
DemandHistory	skuId, locationId, date, unitsSold	R&P, OOS
Vendor	id, name, leadTimeDays	R&P, Reconciliation, DocAI
Carrier	id, name, rateCard, historicalOnTimePct	3PL
PurchaseOrder	id, vendorId, skuId, qty, unitPrice, orderDate, expectedDate	R&P, Reconciliation, DocAI
Shipment	id, carrierId, poId, status, cost, eta, actualArrival	3PL, Reconciliation
GoodsReceipt	id, poId, qtyReceived, receivedDate	Reconciliation
Invoice	id, poId, vendorId, qty, unitPrice, invoiceDate, documentSourceId	Reconciliation, DocAI
Document	id, type (PO/Invoice/BOL), rawFileRef, extractedFields, confidence	DocAI
Transfer	id, skuId, fromLocationId, toLocationId, qty, estCost, status	Stock Transfer
SalvageCandidate	skuId, locationId, ageDays, daysOfSupply, score, recommendedChannel, estRecoveredValue	Salvage
AlertFlag	id, module, entityRef, severity, title, explanation, dollarImpact, recommendedAction, createdAt	All modules → Control Tower home

4. Sample / Synthetic Data Strategy

The single highest-risk element of a pitch demo is data that doesn't cooperate. The generator must be deterministic and scripted, not randomly hoping an interesting pattern emerges.

Parameter	Demo setting	Why
Scale	~70 SKUs · 6 locations · 4 vendors · 4 carriers · 90 days of demand history	Large enough to look real on charts; small enough to stay legible and to keep generation/compute instant.
Seeding	Fixed random seed, versioned in the generator script	Every reset produces the exact same demo-ready scenario — no "the AI found nothing this time."
Planted scenarios	At least: 3 SKUs trending into stockout, 2 locations with a covering transfer opportunity on the same SKU pair, 4 aging/excess SKUs for Salvage, 4 document sets with 1–2 of them deliberately mismatched	Guarantees every module has something true and interesting to show, every single run.
Reset control	A "Reset Demo Data" action available from the UI (see UI Plan §4)	Lets the founder re-run the exact scenario between pitches without a terminal.
Labeling	A persistent "SAMPLE DATA" indicator visible on every screen	Matches the honesty commitment in Business Plan §10 and avoids any ambiguity with diligence-stage investors.

5. Per-Module Technical Approach

5.1 3PL Control

Approach: Deterministic scoring: `score = w1·onTimePct − w2·normalizedCost − w3·claimsRate` computed per carrier from seeded rate cards and shipment history; "best carrier for this shipment" is the highest score among carriers that service the lane.

Inputs: Carrier, Shipment history. **Outputs:** Carrier scorecard, one rate-shopping recommendation, SLA-breach AlertFlag when on-time % drops below a threshold.

CONSTRAINTS

No live carrier/rate API; weights are fixed constants documented in code comments so the "how does it decide" question has a one-sentence answer.

5.2 Replenishment & Planning

Approach: Forecast = simple exponential smoothing over DemandHistory; Safety stock = `z × σ(demand) × √(leadTimeDays)` at a configurable service-level z-score; Reorder point = forecasted demand over lead time + safety stock; suggested PO qty = reorder-up-to-level minus current InventoryPosition.

Inputs: DemandHistory, InventoryPosition, Vendor lead time. **Outputs:** Reorder recommendation per SKU, forecast chart, AlertFlag when projected position will breach reorder point before the next scheduled review.

CONSTRAINTS

Single-location logic only in the demo (no multi-echelon network optimization); service-level slider must visibly change the recommended order quantity live, since that interactivity is a strong demo beat.

5.3 Document AI

Approach: For each sample document: either (a) run real extraction via OCR + an LLM structured-extraction call against a fixed field schema (PO number, SKU, qty, unit price, dates, vendor), or (b) if the build timeline doesn't allow a reliable live path, pre-extract once and cache the structured JSON result keyed to the document id, replaying it instantly in the demo. Confidence score per field drives the UI's highlighting (low-confidence fields shown in amber for review).

Inputs: Document (PDF). **Outputs:** Extracted fields + confidence, written into PurchaseOrder/Invoice records that Reconciliation (5.7) consumes.

CONSTRAINTS

Demo document set fixed at 10–15 synthetic files generated as clean PDFs (not photographs of paper, to avoid OCR variance on stage); must not depend on live network latency during the pitch — cached fallback is mandatory, not optional.

5.4 Stock Transfer Optimization

Approach: For each SKU, compute surplus locations (InventoryPosition well above their own reorder point) and deficit locations (flagged by OOS Prediction, 5.6). Match pairs with a greedy heuristic: largest deficit matched to nearest/cheapest surplus location first, estimated transfer cost from a simple distance × weight formula, compared against the cost of a new purchase order for the same quantity. Recommend the transfer when it is cheaper and faster than reordering.

Inputs: InventoryPosition across locations, OOS risk list. **Outputs:** Ranked transfer recommendations, $ saved vs. new purchase, AlertFlag cleared on the matching OOS SKU once a transfer is approved.

CONSTRAINTS

A full linear-programming solver is optional polish, not required for the demo — the greedy heuristic is sufficient and easier to explain; location count capped at 6 to keep the network view legible (ties to UI Plan).

5.5 Salvage / Liquidation Optimization

Approach: Score = weighted combination of inventory age (days since receipt), days-of-supply at current sell-through, and margin erosion trend; SKUs above a threshold score are ranked and assigned a recommended disposition (discount channel / bundle / liquidation partner / write-off) via a simple rules table keyed on score band and category; estimated recovered value = on-hand units × estimated recovery rate for the assigned channel (a configurable constant per channel, clearly a planning assumption).

Inputs: InventoryPosition, receipt dates, DemandHistory. **Outputs:** Ranked at-risk list, recommended disposition, estimated $ recovered, AlertFlag for newly-crossed-threshold SKUs.

CONSTRAINTS

Recovery-rate constants must be labeled as assumptions in the UI (tooltip), not presented as fact; no live liquidation-marketplace pricing.

5.6 Out-of-Stock Prediction

Approach: Days-to-stockout = current on-hand ÷ recent average daily sell-through (trailing 14-day), adjusted for any known incoming PO/transfer ETA. Risk tier: red if days-to-stockout < lead time with no incoming supply; amber if within 1.5× lead time; green otherwise. Explainability panel renders the three inputs (velocity trend, incoming supply, available transfer) as plain sentences, not a model score.

Inputs: DemandHistory, InventoryPosition, PurchaseOrder ETAs, Transfer availability from 5.4. **Outputs:** Risk-tiered SKU list, countdown, AlertFlag feeding both the home screen and Stock Transfer.

CONSTRAINTS

Sample data generator (Section 4) must guarantee a fixed set of SKUs cross into amber/red — this cannot be left to chance; a heavier ML model (e.g. logistic regression on historical stockout patterns) is a credible Phase 1 upgrade, not required for the demo, and should be described in diligence conversations as the next step rather than implied to already exist.

5.7 Reconciliation with AI Flagging

Approach: For each PO, join its GoodsReceipt and Invoice records; compare quantity, unit price and dates with tolerance thresholds (e.g. flag if price variance > 2% or quantity variance > 0); fuzzy-match on vendor name/invoice number where exact IDs are missing (useful once DocAI-extracted documents, with their own minor OCR noise, are the input). Each mismatch becomes a severity-coded AlertFlag (critical: quantity or price variance above threshold; warning: missing receipt or duplicate invoice number; info: date variance only).

Inputs: PurchaseOrder, GoodsReceipt, Invoice (the latter populated by Document AI, 5.3). **Outputs:** 3-way match table, severity-coded exceptions, drill-down diff, AlertFlag per exception.

CONSTRAINTS

At least 3–4 seeded mismatches required in sample data (Section 4) so flagging is reliably demonstrated; matching thresholds must be visible/configurable in the UI so the logic is not a black box.

6. Shared AI Alerts Engine

Every module writes to one `AlertFlag` table using the schema in Section 3. The Control Tower home screen is simply a query over this table (sorted by severity and dollarImpact), with click-through to the originating module and entity. This single engine is what turns seven separate feature demos into one coherent "control tower" story, and it should be one of the first things built, not the last — every module should be writing into it from day one of its own build, even with placeholder logic.

7. Integration Layer (demo vs. future)

Demo (Phase 0)	Future (Phase 1–2, per Business Plan §9)
Seeded SQLite data spine, generated once, reset on demand	Real connectors: Shopify/NetSuite/SAP for orders & inventory, carrier APIs for 3PL, EDI for vendor documents
CSV import stub (accepts a file, maps columns, loads into the spine) — nice-to-have, not required for the pitch demo	Authenticated, tenant-isolated API connectors with error handling and sync monitoring
No auth required (single-user local/demo instance)	Multi-tenant auth, role-based access, audit log

8. Non-Functional Constraints

- **Offline-first on pitch day:** the demo must run fully on a local laptop with no dependency on live external APIs, cloud database, or Wi-Fi; any optional live-LLM feature (5.3) needs a cached fallback.
- **Fast:** every screen loads in under 2 seconds against the local SQLite file; no spinner longer than a beat during the scripted demo walkthrough.
- **Visibly sample data:** a persistent on-screen indicator (Section 4) and no real PII, customer, supplier or financial data anywhere in the build.
- **Deterministic:** fixed random seed; "Reset Demo Data" always reproduces the same scenario.
- **Single codebase, single deploy target:** no microservices, no second runtime, for exactly the reasons in Section 1.
- **Explainable by design:** every AI flag must be traceable to a plain-language reason a non-technical investor can read in one sentence (ties directly to Business Plan §10's "is this real AI" risk).

9. Build Sequence for Claude Code (solo founder, coding in Claude Code)

Since the founder will use Claude Code for all implementation and UI work, the build should proceed in small, self-contained sprints, each one leaving the app in a demo-able (if incomplete) state. Feed each sprint this document plus the matching section of the UI/UX Plan as the brief.

Sprint	Deliverable	Depends on
0	Scaffold: Next.js + Tailwind + shadcn/ui app shell, nav structure (UI Plan §4), empty data spine schema (Section 3)	—
1	Sample-data generator (Section 4) + seed script + "Reset Demo Data" action	Sprint 0
2	Shared AlertFlag engine (Section 6) + Control Tower home screen reading an empty/placeholder feed	Sprint 1
3	Replenishment & Planning module (5.2) — first real module, proves the pattern	Sprint 2
4	Out-of-Stock Prediction (5.6) + Stock Transfer Optimization (5.4) together — these two are designed to cross-reference each other	Sprint 3
5	Salvage/Liquidation Optimization (5.5)	Sprint 4
6	Document AI (5.3) + Reconciliation (5.7) together — Reconciliation consumes DocAI's output	Sprint 5
7	3PL Control (5.1) — least cross-dependent, safe to build last	Sprint 6
8	Demo-mode polish: guided walkthrough, big-screen mode, presenter notes (UI Plan §7), rehearsal pass against Business Plan §5's scripted beats	All above

CONSTRAINT

Do not start Sprint 8 polish until all seven modules write at least a placeholder AlertFlag — the "one control tower" demo beat depends on every module participating in the shared feed, even before each module's own screen is fully polished.

10. Appendix: Data Dictionary Quick Reference

Full field-level detail belongs in code (Zod schemas); this is the planning-level reference. See Section 3 for entity list and Section 5 for the formulas each module runs against it.

Companion documents: Business Plan & Demo Narrative (document 1) defines why each module exists and the KPIs it must show; UI/UX & Demo Design (document 3) defines how each module's screen is laid out and how the demo-mode experience works on stage.